

BANCO DE DADOS – SEGURANÇA

Profa. Vandecia Fernandes – BiCT/ECP/UFMA

Referências: Elmasri, R. and Navathe, S.B. Sistemas de Bancos de Dados. Korth, H.F. e Silberschatz, A. Sistemas de Bancos de Dados. Notas de aula dos professores: Claudio Baptista, Carlos Eduardo Portela, Ricardo Annes e Higor Medeiros – Artigo devMedia



SEGURANÇA

- Segurança em Banco de dados diz respeito à proteção do banco de dados contra ataques intencionais ou não intencionais, utilizando-se ou não de meios computacionais.

SEGURANÇA

- Os dados armazenados em Bancos de Dados precisam ser protegidos contra acessos não-autorizados, destruição ou alteração intencional e introdução acidental de inconsistências.

SEGURANÇA

- Existem alguns aspectos a serem considerados sobre segurança:
- Aspectos legais, sociais e éticos
- Controles físicos
- Questões políticas
- Problemas operacionais
- Controle de hardware
- Segurança do sistema operacional

SEGURANÇA

- Com a finalidade de proteger o BD, medidas de segurança devem ser tomadas em diversos níveis:
 - Físico
 - Humano
 - Sistema Operacional
 - Sistema de Banco de Dados

SEGURANÇA



Físico – o sistema de computador deve ser fisicamente seguro contra entradas clandestinas de intrusos;

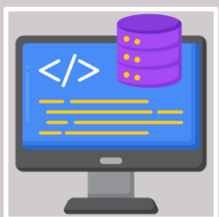


Humano – os usuários devem ser cuidadosamente autorizados, para reduzir a chance de qualquer fornecimento de acesso a um intruso em troca de suborno ou favores;

SEGURANÇA



Sistema operacional – independente da segurança do SGBD, a debilidade na segurança do SO pode servir como um meio de acesso não-autorizado ao banco de dados;



Sistema de BD – alguns usuários de sistemas de BD podem ter autorização de acesso somente a uma porção limitada do BD. Outros usuários podem estar habilitados a emitir consultas, mas podem ser proibidos de modificar dados;

SEGURANÇA

- Principais ameaças à segurança de Bancos de Dados
- 1. Vazamento de Dados:
 - Exposição acidental de informações sensíveis
 - Ataques direcionados para roubo de dados
- 2. Abuso de Privilégios:
 - Uso indevido de credenciais legítimas com altos privilégios
 - Ameaças internas

SEGURANÇA

- Principais ameaças à segurança de Bancos de Dados

3. Acesso Não Autorizado:

- Credenciais fracas ou comprometidas
- Configurações inadequadas de permissões

SEGURANÇA

- Principais ameaças à segurança de Bancos de Dados
- 4. DDoS (Ataque de Negação de Serviço):
 - Sobrecarga do servidor de banco de dados tornando-o indisponível
- 5. SQL Injection:
 - Ataque onde código malicioso é inserido em consultas
 - Pode permitir acesso não autorizado, vazamento ou destruição de dados

SEGURANÇA

- No contexto de bancos de dados, **autenticação** e **autorização** são mecanismos fundamentais para garantir a **segurança dos dados**, controlando quem pode **acessar** e **o que pode fazer** dentro do sistema.

SEGURANÇA – AUTENTICAÇÃO

- Processo de **verificar a identidade de um usuário**. É a etapa onde o sistema confirma se o usuário é quem diz ser.
- Pergunta: Quem é você?

SEGURANÇA – AUTORIZAÇÃO

- A autorização ocorre depois da autenticação e define o nível de acesso que o usuário autenticado terá no banco de dados.
- Pergunta: O que você tem permissão para fazer?"

SEGURANÇA – AUTORIZAÇÃO

- Um usuário pode ter diferentes formas de autorizações sobre o BD:
 - Autorização de leitura
 - Autorização de inserção
 - Autorização de atualização
 - Autorização de eliminação
 - ...
 - Autorização a nível de conta ou relação.

SEGURANÇA – AUTORIZAÇÃO

- Mecanismos de Autorização em Bancos de Dados
- Controle de acesso: Prevenir contra pessoas não autorizadas
 1. Controle de Acesso Discricional (DAC)
 2. Controle de Acesso Mandatário (MAC)

CONTROLE DE ACESSO DISCRICIONÁRIO (DAC)

- Controle de Acesso Discretionário (*DAC - Discretionary Access Control*)
- Controle feito pelo dono dos dados. Ou seja, o proprietário de um recurso (por exemplo, uma tabela) decide quem pode acessar e quais operações podem ser feitas (leitura, escrita, exclusão...).

CONTROLE DE ACESSO DISCRICIONÁRIO (DAC)

- Características:
 - Baseado em privilégios e permissões.
 - O dono do objeto tem discricção (liberdade) para conceder ou revogar acesso.
 - É o modelo mais comum em bancos de dados comerciais.
 - Comandos típicos: GRANT, REVOKE

CONTROLE DE ACESSO DISCRICIONÁRIO (DAC)

VANTAGENS

- Flexível e fácil de administrar.
- Útil para ambientes colaborativos.

DESVANTAGENS

- Pode ser menos seguro, pois usuários autorizados podem repassar privilégios a outros (se permitido).
- Difícil de auditar em sistemas grandes

CONTROLE DE ACESSO MANDATÓRIO (MAC)

- Controle de Acesso Mandatário (MAC - *Mandatory Access Control*)
- Controle feito com base em políticas de segurança definidas pelo sistema, não pelos usuários. Cada dado e cada usuário recebe uma classificação de segurança (por exemplo: público, confidencial, secreto).
- Exemplo:
 - Um documento tem nível "SECRETO". Um usuário com nível "CONFIDENCIAL" não pode acessar, mesmo que seja o dono do banco

CONTROLE DE ACESSO MANDATÓRIO (MAC)

- Características:
- Muito usado em sistemas militares, governamentais e críticos, onde a segurança é prioridade.
- Os usuários não podem alterar as permissões.
- O acesso depende da comparação entre o nível de segurança do usuário e do recurso.

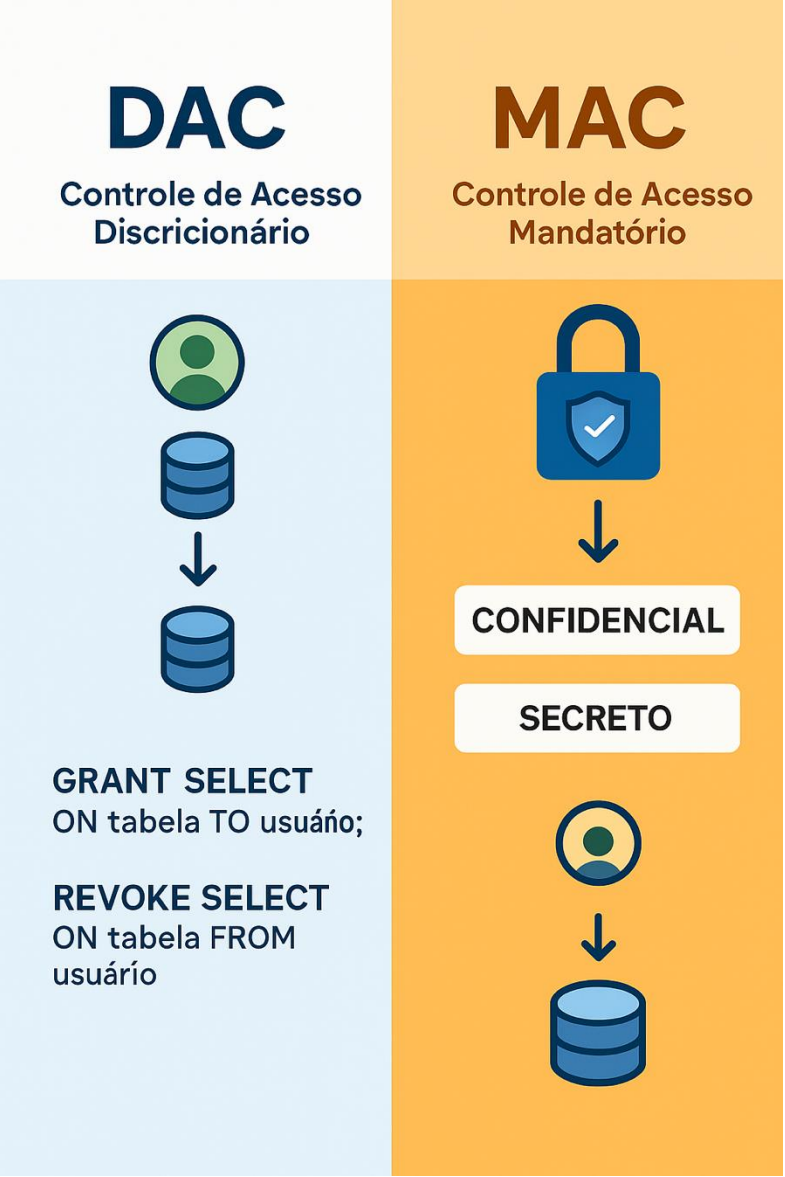
CONTROLE DE ACESSO MANDATÓRIO (MAC)

VANTAGENS

- Muito seguro.
- Evita vazamentos acidentais por usuários.

DESVANTAGENS

- Rígido e mais difícil de gerenciar.
- Pouco flexível para ambientes dinâmicos.



Critério	DAC	MAC
Quem controla o acesso?	Dono do recurso	Administrador do sistema
Flexibilidade	Alta	Baixa (rígido)
Baseado em	Usuários/Grupos	Rótulos/Níveis de segurança
Segurança	Moderada	Alta (para ambientes críticos)
Exemplos	Linux (chmod), SQL (GRANT)	SELinux, Oracle Label Security

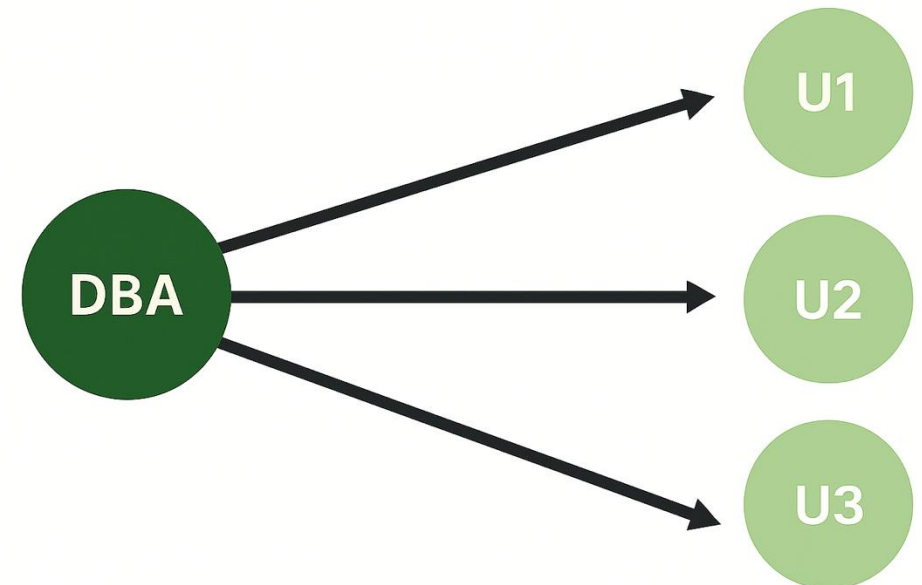
CONCESSÃO DE PRIVILÉGIOS

O DBA fornece/revoga as autorizações aos usuários nas diversas tabelas/visões, e estes podem repassá-los caso recebam autorização para tal.

CONCESSÃO DE PRIVILÉGIOS

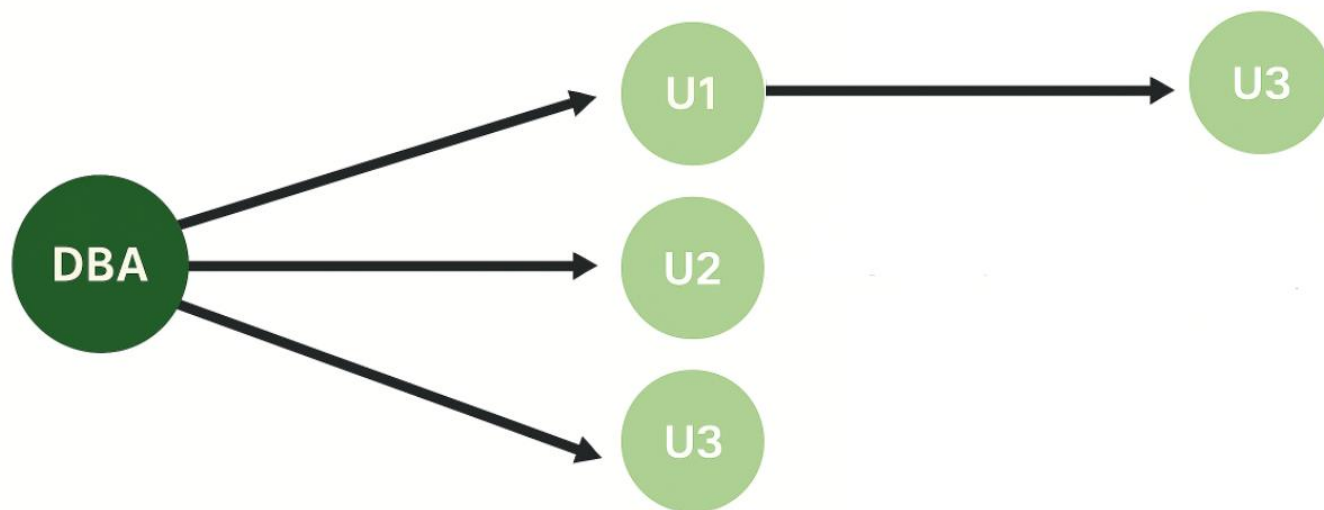
- Considere a concessão de autorização de atualização sobre a relação empréstimo.
- O DBA concede a autorização de atualização a U1, U2 e U3, que podem repassar a sua autorização para outros usuários;

- A passagem de autorização pode ser representada por um grafo de autorizações da seguinte maneira:



CONCESSÃO DE PRIVILÉGIOS

- Todas as arestas de um grafo devem ter um caminho original a partir do DBA:



- Suponhamos que o DBA revogue a autorização de U1;
- U4 teve autorização concedida por U1, sua autorização será revogada também;

PRIVILÉGIOS

- A linguagem SQL inclui comandos para conceder e revogar privilégios. O conjunto de privilégios depende da versão SQL considerada.
- Para criar usuários, usa-se o comando **CREATE USER**:
- Exemplo:
- Criar um usuário sem senha:
 - `CREATE USER <nome do usuário>`

PRIVILÉGIOS

- **Exemplos:**
- Criar um usuário com senha:
 - `CREATE USER <nome do usuário> WITH PASSWORD 'senha'`
- Criar usuários temporários:
 - `CREATE USER <nome do usuário> WITH PASSWORD 'senha' VALID UNTIL '2020-03-09';`

PRIVILÉGIOS

- **GRANT**
- O comando **GRANT** é usado para conferir autorização, sua forma básica é:

GRANT <lista de autorizações>

ON <objeto>

TO <lista de usuários>;

PRIVILÉGIOS

- Exemplos:
 - Concessão de Select sobre a relação funcionários, para os usuários U1, U2:
GRANT SELECT ON funcionarios TO
U1, U2;
 - A autorização update pode ser conferida a todos os campos da relação ou somente a alguns deles:
GRANT UPDATE(nome) ON funcionarios TO
U1, U2;

PRIVILÉGIOS

- Exemplos:
- Permissão de inserção só em algumas colunas:
 - `GRANT INSERT (id, preco, descricao, tipo) ON Produtos TO UI`
- Autorização para o usuário conceder autorização:
 - `GRANT SELECT ON emprestimo TO UI with grant option;`

PRIVILÉGIOS

- Exemplos:
- Instrução ALL PRIVILEGES:
- Permite especificar uma lista de privilégios que inclui todos os privilégios de um objeto específico.
- GRANT ALL PRIVILEGES ON emprestimo TO UI;

PRIVILÉGIOS

- REVOKE
- O comando REVOKE revoga autorização de privilégios
- Sintaxe:
REVOKE <privilégios>
ON <relação/visão>
FROM <usuários>

PRIVILÉGIOS

- Exemplos:
 - REVOKE UPDATE ON Empregado FROM Ana
 - REVOKE ALL PRIVILEGES ON emprestimo FROM UI;
 - REVOKE SELECT ON emprestimo FROM UI, U2, U3 CASCADE

PAPÉIS(ROLES)

Simplificam a **gestão de permissões**, evitando a necessidade de conceder ou revogar privilégios individualmente para cada usuário.

PAPÉIS(ROLES)

- **Papéis (roles)** são uma forma de agrupar **permissões (privilégios)** em um único objeto lógico que pode ser atribuído a um ou mais usuários.
- Exemplo:
 - Gerente, Vendedor, Subgerente

PAPÉIS(ROLES)

- Para criar um papel:
 - `CREATE ROLE <nome-papel>`
- Para apagar o papel:
 - `DROP ROLE <nome-papel>`

PAPÉIS(ROLES)

- Exemplos:
- Criar um papel com nome admin:
 - `CREATE ROLE admin`
- Criar um papel que pode criar bancos de dados e gerenciar papéis:
 - `CREATE ROLE admin WITH CREATEDB CREATEROLE;`

PAPÉIS(ROLES)

- Exemplos:
 - Conceder permissões
 - GRANT SELECT, INSERT ON tabela TO usuario;
 - Revogar permissões
 - REVOKE INSERT ON tabela FROM usuario;
 - Criar roles
 - CREATE ROLE analista;
 - GRANT SELECT ON ALL TABLES TO analista;
 - Atribuir role a usuário
 - GRANT analista TO usuario1;

DDOS

DDoS - Distributed Denial
of Service ou Ataque de
Negação de Serviço
Distribuído

DDOS

- Um ataque de Negação de Serviço Distribuído (DDoS) tem como objetivo sobrecarregar um servidor, serviço ou banco de dados com um volume massivo de requisições, tornando-o indisponível para usuários legítimos.
- Quando direcionado a um banco de dados, o ataque pode causar lentidão extrema, falhas de conexão ou até mesmo a queda completa do sistema.

DDOS

- Geralmente ocorre de três maneiras principais:

I. Sobrecarga de Conexões (*Connection Flood*)

- O atacante utiliza bots (dispositivos infectados) para abrir milhares de conexões simultâneas ao banco de dados.
- Como cada conexão consome recursos (CPU, memória, threads), o servidor fica sobrecarregado e não consegue atender requisições legítimas.

DDOS

2. Ataque de Consultas Lentas (Slow Query Attack)

- O invasor envia consultas SQL maliciosamente complexas que consomem muitos recursos do banco de dados.
- Exemplos:
 - `SELECT * FROM tabela JOIN tabela2 ON ... WHERE I=I`
 - `SELECT COUNT(*) FROM tabela WHERE ... LIKE '%a%'`
- O banco de dados fica travado processando essas consultas, negando serviço a usuários reais.

DDOS

3. Esgotamento de Recursos (Resource Exhaustion)

- O atacante envia transações longas ou bloqueantes que consomem CPU, memória e E/S de disco.
- Exemplo:
 - Transações que travam tabelas (BEGIN; UPDATE tabela SET campo=valor WHERE 1=1;).
 - Consultas que geram alto uso de disco (SELECT * FROM tabela ORDER BY RANDOM()).

SQL INJECTION

SQL Injection – Injeção de
SQL

SQL INJECTION

- O SQL Injection é uma técnica de ataque baseada na manipulação do código SQL, através da entrada de dados em uma página web ou app.
- O ataque de Injeção de SQL consiste de inserções diretas de código nos campos de entrada de dados que serão concatenados com comandos SQL e posteriormente executados.

SQL INJECTION

- A alteração direta de comandos SQL pode expor dados escondidos, sobrescrever dados valiosos, ou ainda executar comandos de sistema perigosos no servidor.

SQL INJECTION

- Exemplo:

```
SELECT * FROM usuarios WHERE username = '[input_usuario]' AND password = '[input_senha]';
```

- Se o invasor digitar:
 - **Usuário:** admin' --
 - **Senha:** (qualquer coisa)
- A consulta se torna:

```
SELECT * FROM usuarios WHERE username = 'admin' --' AND password = '...';
```

- O -- comenta o restante da query, **ignorando a verificação de senha** e permitindo login sem credenciais válidas.

SQL INJECTION

- Exemplo:
- “SELECT * FROM Usuarios WHERE idUsuario = 105 or 1=1”
- SELECT * FROM Usuarios; DROP TABLE Fornecedores

SQL INJECTION

- Recomenda-se:
- Nunca conectar ao banco de dados como super-usuário ou como o dono do banco de dados. Devemos sempre usar usuários personalizados com privilégios limitados.
- Nunca devemos imprimir qualquer informação específica do banco de dados, especialmente sobre o esquema.

SQL INJECTION

- Recomenda-se:
- Sempre verificar se uma entrada qualquer tem o tipo de dados esperado. As linguagens normalmente oferecem funções prontas que já fazem esse tipo de verificação.
- Uso de queries parametrizadas (Prepared Statements)
- Uso de ORM (Object-Relational Mappers)
 - Abstraem o banco de dados em objetos e métodos, evitando a escrita direta de SQL

OI, AQUI É DA ESCOLA
DO SEU FILHO.
ESTAMOS COM UM
PROBLEMA NOS
COMPUTADORES.



MEU DEUS... ELE
ESTRAGOU ALGUMA
COISA?
DE CERTA
FORMA...



ELE REALMENTE
SE CHAMA
Robert*); DROP TABLE
Alunos;-- ?



AH, SIM. NOSSO
PEQUENO BOBBY
TABLES.

BOM, NÓS PERDEMOS OS
REGISTROS DOS ESTUDANTES
DESSE ANO. ESPERO QUE
VOCÊ ESTEJA FELIZ.



E EU ESPERO
QUE VOCÊS
TENHAM
APRENDIDO A
TRATAR AS
ENTRADAS DO
BANCO DE
DADOS

TIRINHA ORIGINAL: [HTTP://XKCD.COM/327](http://xkcd.com/327)

CONCLUSÃO

- A segurança em bancos de dados é multidimensional. Ela depende de:
 - Pessoas bem treinadas e conscientes
 - Processos bem definidos
 - Tecnologias corretamente implementadas



DÚVIDAS?